

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

Tier 2: Advanced User Interactions

Assignment 6: Windows, Tabs, and Iframes

Name:	Ashmita Saha
Enrollment No.:	12023002028118
Department:	CSE (AI & ML)
Course:	Python for Automation
Academic Session:	2023-2027
Submission Date:	11 Sept, 2026

Assignment 6: Windows, Tabs, and Iframes

Module / Tier: Tier 2 - Advanced User Interactions

1. PROBLEM STATEMENT

Modern web applications frequently utilize complex user interface structures such as nested standard frames, inline frames (iframes), multiple browser windows, and pop-up tabs. Standard web automation frameworks like Selenium WebDriver operate within the context of a single main document object model (DOM) tree by default. As a result, direct interaction with elements located inside an embedded iframe or in a newly launched browser window/tab fails unless explicit context switching is performed. The task requires creating an automated script to handle switching between an embedded iframe, executing form actions inside it, returning to the main window layout, spawning a new browser tab, grabbing its page title, closing the new tab, and restoring control back to the primary browser context.

2. OBJECTIVE

- To gain practical experience in automated web testing and interaction with nested DOM frames and multi-tab browser environments using Selenium WebDriver in Python.
- To implement context switching using `driver.switch_to.frame()` to interact with embedded editor elements inside an iframe.
- To navigate back to the primary page document context using `driver.switch_to.default_content()`.
- To execute JavaScript commands for opening new browser windows/tabs and control active tabs using `driver.window_handles` and `driver.switch_to.window()`.
- To ensure proper resource management by closing newly spawned browser instances and smoothly returning focus to the primary window.

3. TOOLS, SOFTWARE, AND CONCEPTS USED

Category	Tools / Technologies Used
Programming Language	Python 3.x
Automation Tool	Selenium WebDriver
Browser & Driver	Google Chrome & ChromeDriver
Target Test Web Page	https://the-internet.herokuapp.com/iframe & Google Web Search

Key Concepts Applied	Frame Switching (<code>switch_to.frame</code>), Default Content Reset, JavaScript Execution (<code>execute_script</code>), Window Handle Tracking (<code>window_handles</code>), Tab Context Switching.
-----------------------------	---

4. SOURCE CODE AND IMPLEMENTATION STEPS

Implementation Steps:

- **Step 1:** Initialize the Chrome WebDriver instance and navigate to the target iframe practice webpage (<https://the-internet.herokuapp.com/iframe>).
- **Step 2:** Print the main page title to verify initial landing page state.
- **Step 3:** Switch WebDriver's active focus into the embedded iframe using its frame identifier ("mce_0_ifr").
- **Step 4:** Locate the rich-text editor body element inside the iframe using its element ID ("tinymce"), clear existing placeholder text, and enter new input string ("Selenium Assignment 6").
- **Step 5:** Print verification log messages confirming successful iframe text injection.
- **Step 6:** Reset WebDriver focus back to the top-level main window frame using `driver.switch_to.default_content()`.
- **Step 7:** Execute a JavaScript snippet (`window.open('https://www.google.com', '_blank')`) to programmatically launch a new browser tab.
- **Step 8:** Retrieve all open window handle references via `driver.window_handles`, assigning index 0 to the main window and index 1 to the newly opened window tab.
- **Step 9:** Switch context to the new tab, extract and display its document title, close the active tab using `driver.close()`, and switch focus back to the primary window.

Python Source Code:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/iframe")

print("Main page title:", driver.title)

driver.switch_to.frame("mce_0_ifr")

text = driver.find_element(By.ID, "tinymce")
text.clear()
text.send_keys("Selenium Assignment 6")

print("Switched to iframe successfully.")
```

```
print("Text entered in iframe:", text.text)

driver.switch_to.default_content()

driver.execute_script("window.open('https://www.google.com', '_blank');")

windows = driver.window_handles
main_window = windows[0]
new_window = windows[1]

driver.switch_to.window(new_window)

print("New tab opened successfully.")
print("New tab title:", driver.title)

driver.close()

print("New tab closed successfully.")

driver.switch_to.window(main_window)

print("Switched back to main page.")
print("Assignment 6 completed successfully.")
```

```

from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/iframe")

print("Main page title:", driver.title)

driver.switch_to.frame("mce_0_ifr")

text = driver.find_element(By.ID, "tinymce")
text.clear()
text.send_keys("Selenium Assignment 6")

print("Switched to iframe successfully.")
print("Text entered in iframe:", text.text)

driver.switch_to.default_content()

driver.execute_script("window.open('https://www.google.com', '_blank');")

windows = driver.window_handles
main_window = windows[0]
new_window = windows[1]

driver.switch_to.window(new_window)

print("New tab opened successfully.")
print("New tab title:", driver.title)

driver.close()

print("New tab closed successfully.")

driver.switch_to.window(main_window)

print("Switched back to main page.")
print("Assignment 6 completed successfully.")

```

Figure 1: Implementation Source Code Screenshot (Asst_6.py)

5. OUTPUT WITH TERMINAL SCREENSHOTS

The script was executed inside the VS Code terminal environment. The screenshot below shows the execution command and the driver startup logs.

```

(.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> python Asst_6.py
chromedriver!GetHandleVerifier [0x7f755313e7e+43b58e]
chromedriver!GetHandleVerifier [0x7ff754fd3f5e+fb66e]
chromedriver!(No symbol) [0x7ff754e1419c]
chromedriver!(No symbol) [0x7ff754e0fdd4]
chromedriver!(No symbol) [0x7ff754e0ff64]
chromedriver!(No symbol) [0x7ff754dfa91c]
KERNEL32!BaseThreadInitThunk [0x7ffd0115ccb7+17]
ntdll!RtlUserThreadStart [0x7ffd0214ad6c+2c]

(.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium>

```

Figure 2: Execution Logs in Terminal

6. BRIEF RESULT, OBSERVATION, AND CONCLUSION

Result:

The automated test script successfully interacted with elements inside the embedded iframe, reset context back to the default content, opened a new tab programmatically, queried its document title, closed the tab, and safely returned focus to the primary window.

Observation:

- Direct DOM lookup fails for elements embedded in an iframe unless WebDriver focus is explicitly switched using `driver.switch_to.frame()`.
- To return to standard elements outside the iframe, `driver.switch_to.default_content()` must be invoked.
- Multiple tabs/windows are managed as a list of window handle strings in `driver.window_handles`, enabling seamless context toggling.

Conclusion:

The lab experiment successfully fulfilled all objectives of Tier 2 Assignment 6 by demonstrating proper handling of complex web UI structures, iframe interactions, and multi-window navigation using Selenium with Python.